

透過 Cloud Run 服務，在 Cloud Storage 使用 Vertex AI Search 搜尋 PDF (非結構化資料)

來源：<https://codelabs.developers.google.com/codelabs/how-to-query-vertex-ai-search-cloud-run-service?hl=zh-tw>

步驟數：8

瞭解如何透過 Cloud Run 服務向 Vertex AI Search 查詢。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 啟用 API](#)
4. [4. 建立搜尋 Cloud Storage 非結構化資料的應用程式](#)
5. [5. 建立 Cloud Run 服務](#)
6. [6. 呼叫 Cloud Run 服務](#)
7. [7. 恭喜！](#)
8. [8. 清除所用資源](#)

1. 簡介

總覽

[Vertex AI Search and Conversation](#) (舊稱 Generative AI App Builder) 可讓開發人員運用 Google 的基礎模型、搜尋專業知識和對話式 AI 技術，建構企業級生成式 AI 應用程式。本程式碼研究室著重於使用 Vertex AI Search，您可運用自有資料建構品質媲美 Google 的搜尋應用程式，並在網頁或應用程式中嵌入搜尋列。

[Cloud Run](#) 是代管運算平台，能讓您在 Google 可擴充的基礎架構上直接執行容器。您可以使用[以原始碼為基礎的部署](#)選項，在 Cloud Run 上部署以任何程式設計語言編寫的程式碼 (可放入容器中)。

在本程式碼研究室中，您將使用以來源為基礎的部署作業建立 Cloud Run 服務，從 Cloud Storage bucket 中的 PDF 檔案擷取非結構化內容的搜尋結果。如要進一步瞭解如何擷取非結構化內容，請參閱[這篇文章](#)。

課程內容

- 如何為從 Cloud Storage 值區擷取的 PDF 非結構化資料，建立 Vertex AI Search 應用程式
 - 如何在 Cloud Run 中使用以來源為基礎的部署作業建立 HTTP 端點
 - 如何根據最小權限原則建立服務帳戶，供 Cloud Run 服務用來查詢 Vertex AI Search 應用程式
 - 如何叫用 Cloud Run 服務來查詢 Vertex AI Search 應用程式
-

步驟 2 · 約 0 分鐘

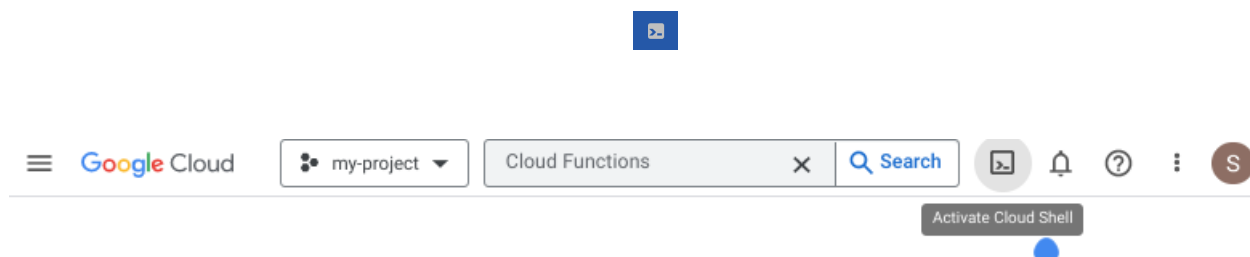
2. 設定和需求

必要條件

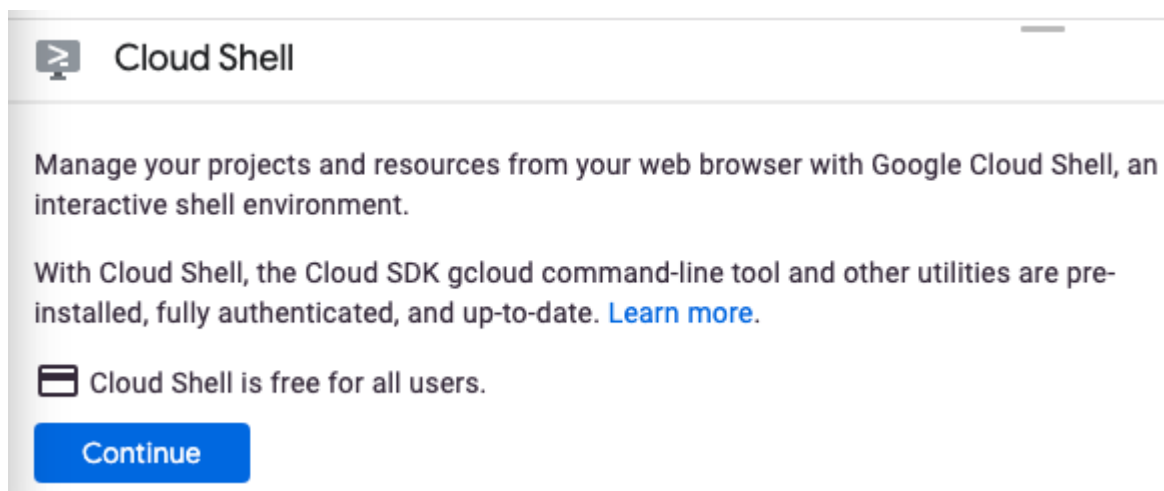
- 您已登入 Cloud Console。
- 您先前已部署 Cloud Run 服務。舉例來說，您可以按照[從原始碼部署網路服務的快速入門導覽課程](#)，開始使用 Cloud Run。

啟用 Cloud Shell

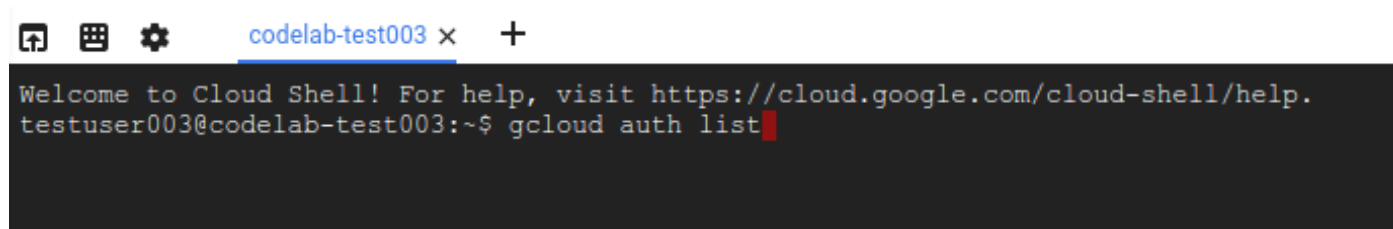
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 0 分鐘

3. 啟用 API

使用 Vertex AI Search 前，請先啟用下列 API。

首先，這個程式碼實驗室需要使用 Vertex AI Search and Conversation、BigQuery 和 Cloud Storage API。您可以在[這裡](#)啟用這些 API。

接著，請按照下列步驟啟用 Vertex AI Search and Conversation API：

1. 在 Google Cloud 控制台中，前往 [Vertex AI Search and Conversation 控制台](#)。
 2. 詳閱並同意《服務條款》，然後點選「Continue and activate the API」。
-

4. 建立搜尋 Cloud Storage 非結構化資料的應用程式

1. 前往 Google Cloud 控制台的「Search & Conversation」(搜尋與對話) 頁面。按一下「New app」(新增應用程式)。
 2. 在「Select app type」(選取應用程式類型) 窗格中，選取「Search」(搜尋)。
 3. 請確認「Enterprise features」(Enterprise 功能)已啟用，這樣系統就會從文件中逐字擷取答案。
 4. 請確認已啟用「進階 LLM 功能」選項，才能取得搜尋摘要。
 5. 在「App name」(應用程式名稱) 欄位中輸入應用程式名稱。應用程式 ID 會顯示在應用程式名稱下方。
 6. 選取「global (Global)」做為應用程式的位置，然後點選「繼續」。
 7. 在「Data stores」(資料儲存庫) 窗格中，點選「Create new data store」(建立新的資料儲存庫)。
 8. 在「Select a data source」(選取資料來源) 窗格中，選取「Cloud Storage」。
 9. 在「Import data from GCS」(從 GCS 匯入資料) 窗格中，確認已選取「Folder」(資料夾)。
 10. 在 **gs://** 欄位中，輸入下列值：`cloud-samples-data/gen-app-builder/search/stanford-cs-224` 這個 Cloud Storage bucket 包含公開 Cloud Storage 資料夾中的 PDF 檔案，僅供測試。
 11. 選取「Unstructured documents」(非結構化文件)，然後按一下「Continue」(繼續)。
 12. 在「設定資料儲存庫」窗格，選取「global (Global)」做為資料儲存庫的位置。
 13. 輸入資料儲存庫的**名稱**。稍後在本程式碼研究室中部署 Cloud Run 服務時，您會使用這個名稱。點選「Create」(建立)。
 14. 在「Data stores」(資料儲存庫) 窗格中，選取新的資料儲存庫，然後點選「Create」(建立)。
 15. 在資料儲存庫的「Data」(資料) 頁面中，點選「Activity」(活動) 分頁標籤，即可查看資料擷取狀態。匯入程序完成後，「狀態」欄會顯示「匯入完成」。
 16. 按一下「Documents」(文件) 分頁標籤，即可查看匯入的文件數量。
 17. 在導覽選單中，點選「預覽」來測試搜尋應用程式。
 18. 在搜尋列中輸入 `final lab due date`，然後按下 **Enter** 鍵，即可查看相關結果。
-

步驟 5 · 約 0 分鐘

5. 建立 Cloud Run 服務

在本節中，您將建立 Cloud Run 服務，接受搜尋字詞的查詢字串。這項服務會使用 [Discovery Engine API](#) 的 Python 用戶端程式庫。如需其他支援的執行階段，請[按這裡查看清單](#)。

建立函式的原始碼

首先，請建立目錄並 cd 到該目錄。

```
mkdir docs-search-service-python && cd $_
```

接著，請建立 `requirements.txt` 檔案，並加入以下內容：

```
blinker==1.6.3
cachetools==5.3.1
certifi==2023.7.22
charset-normalizer==3.3.0
click==8.1.7
Flask==3.0.0
google-api-core==2.12.0
google-auth==2.23.3
google-cloud-discoveryengine==0.11.2
googleapis-common-protos==1.61.0
grpcio==1.59.0
grpcio-status==1.59.0
idna==3.4
importlib-metadata==6.8.0
itsdangerous==2.1.2
Jinja2==3.1.2
MarkupSafe==2.1.3
numpy==1.26.1
proto-plus==1.22.3
protobuf==4.24.4
pyasn1==0.5.0
pyasn1-modules==0.3.0
requests==2.31.0
rsa==4.9
urllib3==2.0.7
Werkzeug==3.0.1
zipp==3.17.0
```

接著，請建立 `main.py` 來源檔案，並加入下列內容：


```

from typing import List
import json
import os
from flask import Flask
from flask import request

app = Flask(__name__)

from google.api_core.client_options import ClientOptions
from google.cloud import discoveryengine_v1 as discoveryengine

project_id = os.environ.get('PROJECT_ID')
location = "global" # Values: "global", "us", "eu"
data_store_id = os.environ.get('SEARCH_ENGINE_ID')

print(project_id)
print(data_store_id)

@app.route("/")
def search_storage():

    search_query = request.args.get("searchQuery")

    result = search_sample(project_id, location, data_store_id, search_query)
    return result

def search_sample(
    project_id: str,
    location: str,
    data_store_id: str,
    search_query: str,
) -> str:
    # For more information, refer to:
    # https://cloud.google.com/generative-ai-app-
builder/docs/locations#specify_a_multi-region_for_your_data_store
    client_options = (
        ClientOptions(api_endpoint=f"{location}-discoveryengine.googleapis.com")
        if location != "global"
        else None
    )

    # Create a client
    client = discoveryengine.SearchServiceClient(client_options=client_options)

    # The full resource name of the search engine serving config
    # e.g.
    projects/{project_id}/locations/{location}/dataStores/{data_store_id}/servingConfigs/
    {serving_config_id}

```

```

serving_config = client.serving_config_path(
    project=project_id,
    location=location,
    data_store=data_store_id,
    serving_config="default_config",
)

# Optional: Configuration options for search
# Refer to the `ContentSearchSpec` reference for all supported fields:
#
https://cloud.google.com/python/docs/reference/discoveryengine/latest/google.cloud.discoveryengine\_v1.types.SearchRequest.ContentSearchSpec
content_search_spec = discoveryengine.SearchRequest.ContentSearchSpec(
    # For information about snippets, refer to:
    # https://cloud.google.com/generative-ai-app-builder/docs/snippets
    snippet_spec=discoveryengine.SearchRequest.ContentSearchSpec.SnippetSpec(
        return_snippet=True
    ),
    # For information about search summaries, refer to:
    # https://cloud.google.com/generative-ai-app-builder/docs/get-search-summaries
    summary_spec=discoveryengine.SearchRequest.ContentSearchSpec.SummarySpec(
        summary_result_count=5,
        include_citations=True,
        ignore_adversarial_query=True,
        ignore_non_summary_asking_query=True,
    ),
)

# Refer to the `SearchRequest` reference for all supported fields:
#
https://cloud.google.com/python/docs/reference/discoveryengine/latest/google.cloud.discoveryengine\_v1.types.SearchRequest
request = discoveryengine.SearchRequest(
    serving_config=serving_config,
    query=search_query,
    page_size=10,
    content_search_spec=content_search_spec,
    query_expansion_spec=discoveryengine.SearchRequest.QueryExpansionSpec(
        condition=discoveryengine.SearchRequest.QueryExpansionSpec.Condition.AUTO,
    ),
    spell_correction_spec=discoveryengine.SearchRequest.SpellCorrectionSpec(
        mode=discoveryengine.SearchRequest.SpellCorrectionSpec.Mode.AUTO
    ),
)

```

```
response = client.search(request)

return response.summary.summary_text

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=int(os.environ.get("PORT", 8080)))
```

設定環境變數

在這段程式碼中，您將建立幾個環境變數，提升本程式碼研究室所用 `gcloud` 指令的可讀性。

```
PROJECT_ID=$(gcloud config get-value project)

SERVICE_NAME="search-storage-pdfs-python"
SERVICE_REGION="us-central1"

# update with your data store name
SEARCH_ENGINE_ID=<your-data-store-name>
```

建立服務帳戶

這堂程式碼實驗室會說明如何為 Cloud Run 服務建立服務帳戶，以便存取 Vertex AI Search API。

```
SERVICE_ACCOUNT="cloud-run-vertex-ai-search"
SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT@$PROJECT_ID.iam.gserviceaccount.com

gcloud iam service-accounts create $SERVICE_ACCOUNT \
  --display-name="Cloud Run Vertex AI Search service account"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \
  --role='roles/discoveryengine.editor'
```

部署 Cloud Run 服務

您現在可以使用以來源為基礎的部署作業，自動將 Cloud Run 服務容器化。

```
gcloud run deploy $SERVICE_NAME \
  --region=$SERVICE_REGION \
  --source=. \
  --service-account $SERVICE_ACCOUNT \
  --update-env-vars SEARCH_ENGINE_ID=$SEARCH_ENGINE_ID,PROJECT_ID=$PROJECT_ID \
  --no-allow-unauthenticated
```

然後將 Cloud Run 網址儲存為環境變數，以供後續使用。

```
ENDPOINT_URL="$(gcloud run services describe $SERVICE_NAME --region=$SERVICE_REGION -  
-format='value(status.url)')"
```

步驟 6 · 約 0 分鐘

6. 呼叫 Cloud Run 服務

現在可以透過查詢字串呼叫 Cloud Run 服務，向 `What is the final lab due date?` 提問。

```
curl -H "Authorization: bearer $(gcloud auth print-identity-token)" "$ENDPOINT_URL?searchQuery=what+is+the+final+lab+due+date"
```

結果應類似下方的輸出範例：

```
The final lab is due on Tuesday, March 21 at 4:30 PM [1].
```

步驟 7 · 約 0 分鐘

7. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱 [Vertex AI Search](#) 和 [Cloud Run](#) 的說明文件。

涵蓋內容

- 如何為從 Cloud Storage 值區擷取的 PDF 非結構化資料，建立 Vertex AI Search 應用程式
 - 如何在 Cloud Run 中使用以來源為基礎的部署作業建立 HTTP 端點
 - 如何遵循最小權限原則，為 Cloud Run 服務建立服務帳戶，以便查詢 Vertex AI Search 應用程式。
 - 如何叫用 Cloud Run 服務來查詢 Vertex AI Search 應用程式
-

8. 清除所用資源

為避免產生非預期費用 (例如，如果這個 Cloud 函式遭非預期地叫用次數超過[免費層級的每月 Cloud 函式叫用配額](#))，您可以刪除 Cloud 函式，或刪除您在步驟 2 中建立的專案。

如要刪除 Cloud 函式，請前往 Cloud Function Cloud 控制台 (<https://console.cloud.google.com/functions/>)，然後刪除 **imagen_vqa** 函式 (如果您使用其他名稱，請刪除 \$FUNCTION_NAME)。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
